

Erwin Quanten

Staff Software Engineer –Next.js, NestJS AWS

Brussels, Belgium | erwin.quanten.dev@outlook.com | +32460256356 | <https://www.linkedin.com/in/erwin-quanten-b82815a/>

PROFESSIONAL SUMMARY

Staff Software Engineer with 16+ years building **AI SaaS**, investment intelligence, telecom, manufacturing, and B2B automation products using **TypeScript 5.x**, **Next.js 15**, **React 19**, **NestJS 10.x**, **Prisma 5.x**, **PostgreSQL 14–16**, **AWS EKS 1.28–1.30**, **Terraform 1.x**, and Docker. Experienced designing LLM-first platforms with **OpenAI**, **Claude**, **Gemini**, RAG ingestion, structured outputs, tool calling, async job queues, web scraping orchestration, and multi-tenant RBAC, while migrating Vercel/Supabase-style workloads to secure AWS infrastructure with faster releases, stronger observability, predictable costs, and safer deployments.

SKILLS

- **Frontend: React 18–19, Next.js 14–15**, App Router, Server Components, Server Actions, Route Handlers, TypeScript 5.x, Tailwind CSS 3.x–4.x, React Query, Zustand, Redux Toolkit, Zod, WebSockets
- **Backend: Node.js 18–22, NestJS 9–10**, Express.js 4.x, REST APIs, GraphQL basics, Prisma 4–5, PostgreSQL 14–16, Redis, JWT/JWKS, RBAC, cron jobs, dependency injection
- **AI / LLM: OpenAI API, Anthropic Claude, Google Gemini**, RAG, structured outputs, function calling, tool-use workflows, prompt engineering, embeddings, document Q&A, PDF/DOCX/XLSX ingestion, LangChain, LangGraph
- **Data / Automation: Apify, Puppeteer**, proxy-based scraping, async queues, BullMQ, retry handling, status tracking, LLM extraction pipelines, report generation, sentiment analysis, persona generation
- **AWS / DevOps: AWS EKS**, RDS PostgreSQL, S3, ECR, IAM, ALB Ingress, CloudWatch, CloudTrail, Secrets Manager, SSM Parameter Store, SES, Cognito, Docker, Kubernetes, Helm, Terraform, GitHub Actions

EXPERIENCE

Lightedge — Lead Software Engineer

Remote/ Full time

May2022 – Apr 2026

Houston, TX

- Built a production **AI SaaS** platform with **TypeScript 5.x**, **Next.js 15**, **React 19**, **NestJS 10.x**, **Prisma 5.x**, and **PostgreSQL 15**, supporting multi-tenant customer workflows with org-scoped data access.
- Designed **LLM-powered document intelligence** using **OpenAI**, **Claude**, and **Gemini** APIs, including streaming responses, structured outputs, schema validation with **Zod**, and provider-agnostic model routing.
- Implemented **RAG pipelines** for PDF, DOCX, and XLSX ingestion, combining text extraction, chunking, embeddings, retrieval scoring, and conversational document Q&A for internal analyst workflows.
- Created an **agentic workflow engine** where queued jobs orchestrated scraping, LLM extraction, validation, scoring, and report generation without blocking the user-facing application.
- Reduced failed AI report generation by **31%** by adding retry policies, prompt fallback logic, model-specific timeout handling, and persisted job-state recovery through **BullMQ** and Redis.
- Migrated selected workloads from managed deployment patterns toward **AWS EKS 1.29**, **RDS PostgreSQL**, **S3**, **ECR**, **ALB Ingress**, and **Secrets Manager** to improve infrastructure control and cost visibility.
- Built **Terraform 1.x** modules for VPC, EKS, RDS, IAM, security groups, S3 buckets, and CI-driven plan/apply workflows with environment-specific state management.
- Improved deployment reliability by **27%** through Docker multi-stage builds, Helm chart templating, Kubernetes readiness probes, rolling deployments, and GitHub Actions deployment gates.
- Fixed a critical **JWT authorization bug** where stale tenant claims allowed incorrect workspace access by adding JWKS validation, token rotation checks, and request-level RBAC guards.
- Developed white-label theming with **Tailwind CSS v4**, tenant configuration records, dynamic branding tokens, and reusable React layout primitives for partner-specific product experiences.
- Improved AI extraction accuracy by **34%** by introducing few-shot prompt templates, strict JSON schemas, validation repair prompts, and model comparison dashboards for cost and latency decisions.
- Used **Claude Code**, **Cursor**, **GitHub Copilot**, and manual code review together to accelerate implementation while keeping architecture decisions, security rules, and tests under engineering control.

- Delivered enterprise SaaS applications using **React 16–18**, **TypeScript 3.9–4.8**, **Node.js 14–16**, **NestJS 7–9**, **PostgreSQL 12–14**, Redis, Docker, and AWS services.
- Designed modular **NestJS** services with decorators, guards, middleware, cron jobs, dependency injection, and layered validation for customer-facing workflow automation platforms.
- Built customer analytics and reporting features with React dashboards, REST APIs, PostgreSQL aggregation queries, background workers, and downloadable report generation.
- Reduced API response time by **39%** by replacing heavy ORM joins with optimized PostgreSQL indexes, selective projections, pagination, and cached query results in Redis.
- Migrated legacy JavaScript modules into **TypeScript 4.x** with shared DTOs, stricter compiler options, typed API clients, and safer validation logic across frontend and backend services.
- Implemented authentication and authorization using **JWT**, refresh-token rotation, RBAC permissions, route-level guards, and audit logging for sensitive enterprise workflows.
- Fixed a recurring production issue where duplicate queue messages created repeated reports by adding idempotency keys, transactional status updates, and dead-letter handling.
- Improved release speed by **28%** with GitHub Actions pipelines that ran tests, built Docker images, scanned dependencies, pushed images to ECR, and deployed through Kubernetes manifests.
- Built multi-tenant configuration features that allowed customers to customize branding, feature flags, notification templates, and access rules without separate code branches.
- Reduced manual operations by **35%** by creating scheduled background jobs, admin retry screens, structured logs, and CloudWatch alerts for failed imports and delayed queues.

- Modernized internal manufacturing and logistics applications using **React 15–16**, **AngularJS 1.5**, **Node.js 6–8**, **Express 4.x**, **PostgreSQL 9.6**, Docker, and AWS EC2 services.
- Built operational dashboards for plant, inventory, and supply-chain teams with reusable UI components, role-based screens, REST API integration, and responsive layouts.
- Migrated older jQuery-heavy interfaces into **React 16.x** modules while preserving existing backend routes, reducing rewrite risk and allowing gradual feature-by-feature modernization.
- Improved dashboard load performance by **33%** by reducing unnecessary API calls, caching reference data, adding server-side filtering, and optimizing PostgreSQL query plans.
- Implemented new approval workflows for logistics exceptions, including status history, email notifications, manager permissions, and audit-friendly change tracking.
- Fixed a difficult timezone bug in shipment scheduling by normalizing UTC storage, adding explicit browser timezone conversion, and expanding regression tests around daylight-saving changes.
- Reduced production support tickets by **24%** by improving form validation, user-facing error messages, API exception handling, and logging around failed business-rule checks.
- Introduced Docker-based local environments that helped developers reproduce issues faster and reduced machine-specific configuration problems across frontend and backend services.
- Supported early AWS adoption with EC2, S3, RDS, IAM users, security groups, backup scripts, and basic deployment automation for business-critical internal applications.

- Developed enterprise workflow systems using **AngularJS 1.2–1.4**, **jQuery 1.9–2.x**, **Node.js 0.12–4.x**, **Express 3–4**, Bootstrap 3, REST APIs, and relational databases.
- Created inventory and procurement modules with searchable grids, approval forms, validation rules, and backend APIs that connected business users with operational data.
- Improved report generation time by **26%** by moving expensive calculations from browser-side scripts into optimized backend services and indexed SQL queries.
- Migrated older server-rendered pages into AngularJS-based interfaces, keeping existing authentication and database logic stable while making workflows more interactive.
- Fixed a recurring data mismatch issue between procurement and inventory screens by adding shared validation rules, database constraints, and reconciliation scripts.

- Built scheduled import jobs for supplier and product data, including CSV parsing, duplicate detection, error logging, and admin visibility into failed records.
- Reduced manual spreadsheet work by **22%** through new internal tools that automated approval routing, search filters, status updates, and downloadable operational summaries.
- Worked flexibly across frontend, backend, database, and deployment tasks, helping a small engineering team deliver features without waiting for strict role boundaries.

Tessengerlo Group — Software Developer

Hybrid/ Full time

Jun 2009 – Aug 2011

Brussel Metropolitan Area

- Built business web applications using **JavaScript, jQuery 1.4–1.7**, HTML, CSS, PHP 5.3, early REST-style endpoints, SQL databases, and internal server deployments.
- Implemented reusable form screens, validation logic, table filters, export actions, and admin tools for departments managing operational and customer data.
- Reduced repetitive manual data entry by **18%** by creating browser-based tools that replaced spreadsheet-driven tracking with centralized database-backed workflows.
- Fixed browser compatibility issues in older Internet Explorer environments by adjusting JavaScript event handling, CSS layouts, and progressive enhancement patterns.
- Improved internal page stability by **21%** by adding defensive validation, clearer error messages, safer SQL query handling, and repeatable deployment checklists.

EDUCATION

KU Leuven — Master's Degree, Computer Science

Sep 2003 – May 2009